HDOC Day-57 Activity

Maximum Contiguous Subarray Sum Using Kadane's Algorithm and Pointers

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-57

Aim

To write a C program that finds the maximum sum of any non-empty contiguous subarray using **Kadane's algorithm**. The array is passed to **max_subarray_sum()** using its address and accessed through pointers.

Problem Requirements

- Read the size and elements of a variable-length integer array **arr[]**.
- Pass the array to **max_subarray_sum()** using its address.
- Find the maximum sum of a contiguous subarray using Kadane's algorithm.
- Ensure the selected subarray contains at least one element.
- If every element is negative, return the largest (least negative) element.
- Display the maximum subarray sum.

Kadane's Algorithm

Kadane's algorithm keeps two values: **current_sum**, the best sum of a subarray ending at the current position, and **max_sum**, the best sum found anywhere so far. At each element, we decide whether it is better to start a new subarray from that element or extend the previous subarray.

Algorithm

1. Read the array size **n** and the elements of **arr[]**.

2. Pass **arr** and **n** to **max_subarray_sum(arr, n)**.

3. Initialize both **current_sum** and **max_sum** with the first array element. This guarantees that at least one element is selected and correctly handles all-negative arrays.

4. Traverse from the second element to the end.

5. For each value, compare the value itself with **current_sum + value**. Store the larger one in current_sum.

6. If current_sum becomes greater than max_sum, update max_sum.

7. Display max_sum.

C Program

```
#include <stdio.h>

void max_subarray_sum(int *arr, int n)
{
    int current_sum = *arr;
    int max_sum = *arr;

    for (int i = 1; i < n; i++)
    {
        int value = *(arr + i);

        if (value > current_sum + value)
        {
            current_sum = value;
        }
        else
        {
            current_sum = current_sum + value;
        }

        if (current_sum > max_sum)
        {
            max_sum = current_sum;
        }
    }

    printf("Maximum subarray sum: %d", max_sum);
}

int main()
{
    int n;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements: ");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    max_subarray_sum(arr, n);

    return 0;
}
```

Pointer Usage

max_subarray_sum(arr, n) passes the address of the first element. The parameter **int *arr** receives that address. ***arr** accesses the first element and ***(arr + i)** accesses the element at index i.

Example 1 Dry Run

Value	current_sum	max_sum
-2	-2	-2
1	1	1
-3	-2	1

4	4	4
-1	3	4
2	5	5
1	6	6
-5	1	6
4	5	6

The maximum becomes **6**, corresponding to the contiguous subarray **4 -1 2 1**.

Test Cases / Expected Results

Case	Input Array	Expected Output
Example 1	-2 1 -3 4 -1 2 1 -5 4	Maximum subarray sum: 6
Example 2	-8 -3 -6 -2 -5 -4	Maximum subarray sum: -2
Example 3	5 -2 3 4	Maximum subarray sum: 10

Example 2 - All Negative Elements

The algorithm starts with the first element instead of zero. Therefore, when all values are negative, it still selects at least one element. For **-8 -3 -6 -2 -5 -4**, the largest value encountered is **-2**, so the maximum subarray sum is -2.

Example 3 Explanation

For **5 -2 3 4**, extending the subarray remains beneficial throughout the traversal. The entire array gives **5 + (-2) + 3 + 4 = 10**, which is the maximum.

Complexity

Time Complexity: O(n), because Kadane's algorithm traverses the array only once.

Space Complexity: O(1) extra space because only a few variables are required.

Result

The program successfully finds the maximum sum of a non-empty contiguous subarray using Kadane's algorithm and pointer-based array access. It also correctly handles arrays containing only negative elements by selecting the largest, least negative, element.

Execution Example

```
Enter the size of array: 9
Enter the elements: -2 1 -3 4 -1 2 1 -5 4
Maximum subarray sum: 6
```